# CloudEagle DevOps Assignment

## Executive Summary

This document presents a practical production-ready design for the CloudEagle assignment. The solution keeps the operating model simple while still following strong DevOps practices. For CI/CD, Jenkins Multibranch Pipeline is used so pull requests only validate code, while approved branch merges and release tags control deployments. For infrastructure, the recommended runtime is Google Compute Engine Managed Instance Groups behind a load balancer, with MongoDB Atlas as the managed database layer.

The key goal of this design is to balance safety, cost, and operational simplicity. It avoids accidental production deployments, supports rolling releases with minimal downtime, keeps secrets out of source control, and uses managed GCP services where they reduce long-term maintenance effort.

## Question 1 – Deployment and CI/CD Design

**Scenario: We have a Spring Boot backend service called sync-service. It connects to MongoDB, runs on GCP VMs, has three environments (qa, staging, prod), and is built through Jenkins.**

### 1.1 Recommended Branching Strategy

The cleanest mapping is to use branches for development flow and version tags for production release. This keeps day-to-day work simple and makes production releases traceable.

| Git Reference | Mapped Environment | Pipeline Behavior | Safety Control |
|---|---|---|---|
| feature/* or bugfix/* | None | PR validation only. Never deploy. | No environment access |
| develop | QA | Build, test, push image, deploy automatically. | Protected branch |
| release/* | Staging | Build, test, push image, deploy automatically. | Protected branch |
| main | No direct deploy by default | Used for approved release history. | Protected branch |
| vX.Y.Z tag on main | Production | Manual approval, then deploy. | Protected tags + approval gate |

Why this is good:

- QA stays close to active development.
- Staging behaves like a release-candidate environment.
- Production is linked to a version tag, not a moving branch.
- Rollback is cleaner because every production release maps to a clear version.

### 1.2 How to Avoid Accidental Production Deployments

Production safety should never depend on only one control. The safest design is to combine source-control protection with pipeline protection.

1. Protect the main branch so merges require approvals and passing checks.
2. Protect release tags such as v1.2.0 so only release managers or CI automation can create them.
3. Allow the production deployment stage in Jenkins only when the build is a valid tag build.
4. Add a manual approval step in Jenkins for production deployment.

5. Limit who can approve production rollout to a small release group.

This means an ordinary merge to main will not directly push code to production. A release tag plus approval is required.

## 1.3 Jenkins Pipeline Design

The Jenkins pipeline should be multibranch, stage-based, and environment-aware. It should build the application once, store the artifact or image, and then promote the same version across environments.

| Stage | PR | Merge to develop | Merge to release/* | Release tag |
|---|---|---|---|---|
| Checkout | Yes | Yes | Yes | Yes |
| Build | Yes | Yes | Yes | Yes |
| Unit Tests | Yes | Yes | Yes | Yes |
| Static Analysis | Yes | Yes | Yes | Yes |
| Integration Tests | Yes | Yes | Yes | Yes |
| Build Image | Yes | Yes | Yes | Yes |
| Push Image | No | Yes | Yes | Yes |
| Deploy | No | QA | Staging | Production |
| Smoke Test | No | Yes | Yes | Yes |

PR behavior:

- Checkout code and dependencies.
- Compile the Spring Boot application.
- Run unit tests and basic quality checks.
- Optionally run lightweight integration tests.
- Build the image for validation only.
- Do not deploy anything.

Merge behavior:

- For develop: push the image and deploy to QA.
- For release/*: push the image and deploy to staging.
- For production release tag: wait for manual approval, then deploy to prod.

## 1.4 High-Level Pipeline Stages

6. Checkout source code from Git.
7. Resolve target environment from branch or tag.
8. Build the JAR with Maven.
9. Run tests and quality checks.
10. Build a Docker image using the same application package.
11. Push the immutable image to Artifact Registry.
12. Create or update the Compute Engine instance template.
13. Start rolling update on the Managed Instance Group.
14. Run smoke test on the target environment.
15. Rollback automatically if validation fails.

## 1.5 Rollback Strategy

A safe VM-based rollback strategy is to keep track of the previous instance template and move the Managed Instance Group back to that version if the new rollout fails.

16. Before deployment, record the current instance template as the last known good version.
17. Create a new instance template for the new release.

18. Start a rolling update with surge capacity.
19. Wait until instances become healthy and pass smoke checks.
20. If the checks fail, stop the rollout and start a new rolling update using the previous template.
21. Confirm service health and only then close the incident.

Important note: application rollback is easy only when database changes are backward compatible. For that reason, schema or data changes should follow an expand-migrate-contract approach.

## 1.6 Configuration Management

The best approach is to keep the Docker image immutable and load environment-specific configuration at runtime.

- Use Spring profiles such as qa, staging, and prod.
- Store non-secret config separately for each environment.
- Pass settings like log levels, API endpoints, and feature flags as environment variables or external configuration.
- Do not rebuild the image separately for each environment.

| Configuration Type | Where It Should Live | Examples |
|---|---|---|
| Non-secret environment config | External config / environment variables | Spring profile, log level, service URLs |
| Secrets | Secret Manager | MongoDB credentials, API keys, tokens |
| Build metadata | Artifact Registry / CI metadata | Image tag, commit SHA, build number |

## 1.7 Secrets Handling

Secrets must never be hardcoded in source code, Docker images, Jenkinsfiles, or VM templates.

- Store MongoDB credentials and API keys in Google Secret Manager.
- Use one runtime service account per environment.
- Grant only the minimum secret access needed.
- Fetch secrets at runtime, not at build time.
- Rotate secrets regularly and monitor secret access through audit logs.

## 1.8 Deployment Strategy: Blue/Green vs Rolling vs Recreate

| Strategy | Strength | Weakness | Verdict |
|---|---|---|---|
| Recreate | Very simple | Causes downtime | Not recommended |
| Blue/Green | Fast rollback and low release risk | Needs duplicate capacity and higher cost | Good for rare high-risk changes |
| Rolling | Lower cost, native to Managed Instance Groups, works well with health checks | Rollback is slower than blue/green | Recommended default |

Recommended choice: Rolling deployment.

Why rolling is best here:

- The assignment already uses GCP VMs, so Managed Instance Groups are a natural fit.
- Rolling updates work well with health checks and autoscaling.
- Cost stays lower than always-on blue/green environments.
- Downtime can be kept minimal when maxUnavailable is set to 0 and surge capacity is enabled.

## 1.9 Zero or Minimal Downtime Approach

22. Use a load balancer in front of the service.
23. Enable health checks so only healthy instances receive traffic.
24. Use rolling updates with maxUnavailable=0 and maxSurge greater than 0.
25. Expose readiness and liveness endpoints from Spring Boot.
26. Enable graceful shutdown so current requests can complete before instance termination.
27. Run smoke tests immediately after deployment.

# Question 2 – Infrastructure Design

**Scenario: We need to run the same service on GCP with auto-scaling, secure access, and reasonable cost for a startup.**

## 2.1 Recommended Compute Choice

The strongest recommendation for this assignment is Compute Engine Managed Instance Groups, not GKE and not Cloud Run as the primary answer.

| Option | What It Gives You | Trade-Off | Verdict |
|---|---|---|---|
| Cloud Run | Fully managed runtime and scale-to-zero | Changes the current VM-based operating model | Strong future option |
| GKE | Maximum flexibility for many microservices | More platform complexity and higher ops overhead | Too heavy for this single-service case |
| Compute Engine MIG | Direct fit for VM deployment, autoscaling, autohealing, rolling updates | You still manage VM image/template lifecycle | Recommended |

Why Compute Engine MIG is the best fit:

- It matches the assignment requirement that the service runs on GCP VMs.
- It supports autoscaling, autohealing, and rolling deployments.
- It is simpler than Kubernetes for a single service.
- It keeps startup cost and operational complexity under control.

## 2.2 MongoDB Hosting Approach

The recommended database choice is MongoDB Atlas on Google Cloud.

- Managed backups and point-in-time recovery.
- Lower operational burden compared to self-managed replica sets.
- Private connectivity options for better security.
- Better long-term reliability for a startup team with limited ops bandwidth.

Self-managed MongoDB on VMs is possible, but it adds backup management, patching, failover design, monitoring, and security hardening effort. For this assignment, Atlas is the stronger production answer.

## 2.3 Networking Basics

- Use a separate project per environment where possible, especially for production.
- Create a VPC with regional private subnets.
- Keep application VMs private with no public IPs.
- Use an external or internal HTTPS load balancer depending on exposure requirements.

- Use health checks to send traffic only to healthy VMs.
- Use Cloud NAT for controlled outbound internet access.
- Use Private Google Access for access to Google services from private VMs.
- Use IAP TCP forwarding instead of opening SSH publicly.

This design reduces attack surface and keeps environment isolation clean.

## 2.4 Secrets and IAM

The IAM model should be simple and strict.

- One runtime service account per environment.
- One separate CI/CD deployment identity for Jenkins.
- No broad Editor-style roles in production.
- Least-privilege access for Secret Manager, Artifact Registry, logging, and compute operations.

The application should read secrets only at runtime. Jenkins should have deployment rights, but not unrestricted admin rights across every project.

## 2.5 Logging and Monitoring Stack

- Install the Ops Agent on all application VMs.
- Send logs to Cloud Logging.
- Use Cloud Monitoring dashboards for health, CPU, memory, and latency.
- Create alerts for 5xx spikes, instance health failures, deployment rollback events, and MongoDB connectivity issues.
- Use structured logs so troubleshooting is faster.

This gives enough observability for production without adding unnecessary third-party tools on day one.

## 2.6 Autoscaling and Cost Control

Start with simple and measurable autoscaling rules.

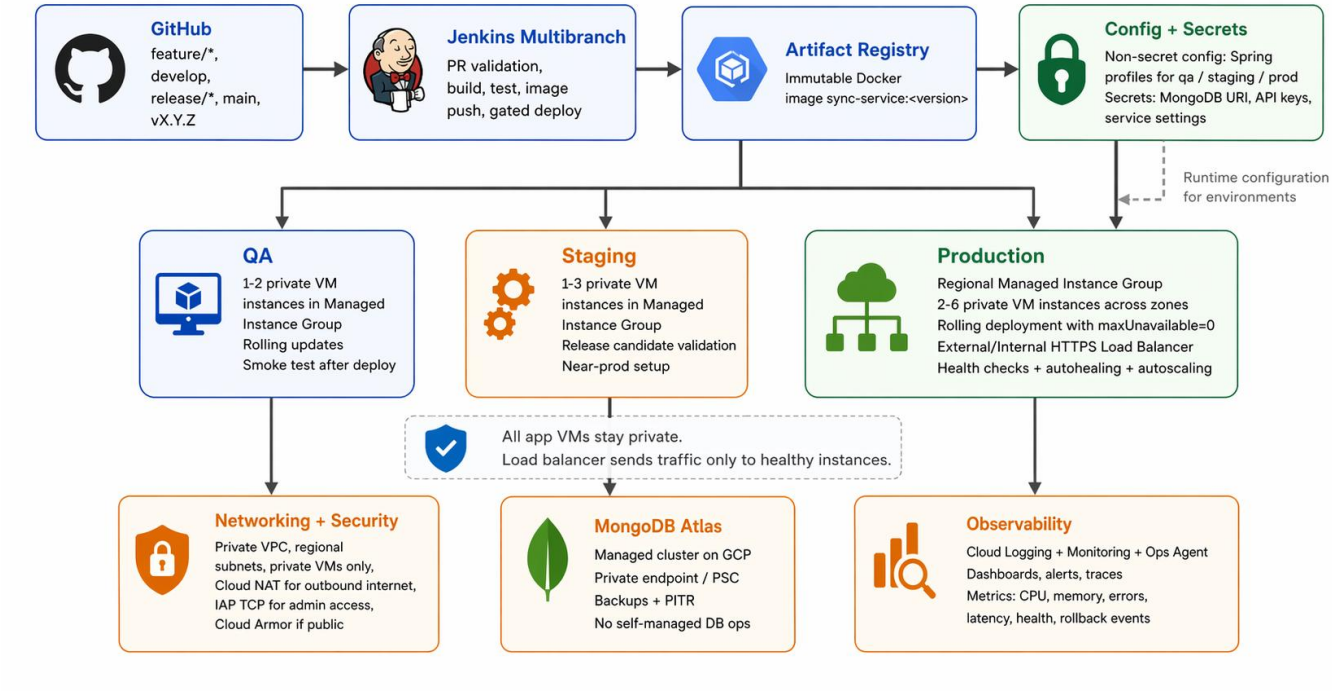
| Environment | VM Type | Min / Max | Notes |
|---|---|---|---|
| QA | e2-standard-2 | 1 / 2 | Cheap and enough for testing |
| Staging | e2-standard-2 | 1 / 3 | Good for release rehearsal |
| Production | e2-standard-4 | 2 / 6 | Better headroom and zone resilience |

These are starting values. They should be tuned after load testing and real production metrics.

Additional cost controls:

- Use autoscaling instead of fixed high instance counts.
- Use Spot VMs only for non-critical workloads such as temporary Jenkins agents.
- Keep QA and staging smaller than production.
- Review rightsizing after the first few weeks of monitoring data.

## 2.7 Recommended Architecture Diagram




**The diagram above shows the recommended flow:** GitHub to Jenkins, Jenkins to Artifact Registry, deployment to Managed Instance Groups, runtime configuration from Secret Manager and Spring profiles, MongoDB Atlas as the managed database, and Cloud Logging / Monitoring for observability.

## Final Recommendation

For this assignment, the best overall answer is:

- Use Jenkins Multibranch Pipeline with protected branches and protected production tags.
- Deploy automatically to QA from develop and to staging from release/*.
- Deploy to production only from signed or approved version tags with a manual approval gate.
- Use Compute Engine Managed Instance Groups with rolling updates and health checks.
- Use MongoDB Atlas on Google Cloud instead of self-managing MongoDB on VMs.
- Use Secret Manager for secrets and least-privilege IAM for access control.
- Use Cloud Logging, Monitoring, and Ops Agent for production observability.

## Appendix A – Sample Jenkinsfile

The following Jenkinsfile is included so the submission contains the required pipeline example.

```
pipeline {
    agent any

    options {
        timestamps()
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '20'))
    }

    environment {
        APP_NAME            = 'sync-service'
```

```
    ARTIFACT_NAME       = 'sync-service.jar'
    GCP_PROJECT_QA      = 'my-qa-project'
    GCP_PROJECT_STAGING = 'my-staging-project'
    GCP_PROJECT_PROD    = 'my-prod-project'
    REGION              = 'asia-south1'
    QA_MIG              = 'sync-service-qa-mig'
    STAGING_MIG         = 'sync-service-staging-mig'
    PROD_MIG            = 'sync-service-prod-mig'
    HEALTH_PATH         = '/actuator/health'
}

stages {
    stage('Checkout') {
        steps {
            checkout scm
            script {
                env.SHORT_COMMIT = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
                env.BUILD_VERSION = "${env.BRANCH_NAME}-${env.BUILD_NUMBER}-${env.SHORT_COMMIT}"
                env.IS_PR = env.CHANGE_ID ? 'true' : 'false'
            }
        }
    }

    stage('Set Target Environment') {
        steps {
            script {
                env.TARGET_ENV = ''
                env.GCP_PROJECT = ''
                env.MIG_NAME = ''

                if (env.IS_PR == 'true') {
                    echo 'Pull request build detected. Validation only.'
                } else if (env.BRANCH_NAME == 'develop') {
                    env.TARGET_ENV = 'qa'
                    env.GCP_PROJECT = env.GCP_PROJECT_QA
                    env.MIG_NAME = env.QA_MIG
                } else if (env.BRANCH_NAME.startsWith('release/')) {
                    env.TARGET_ENV = 'staging'
                    env.GCP_PROJECT = env.GCP_PROJECT_STAGING
                    env.MIG_NAME = env.STAGING_MIG
                } else if (env.BRANCH_NAME == 'main') {
                    env.TARGET_ENV = 'prod'
                    env.GCP_PROJECT = env.GCP_PROJECT_PROD
                    env.MIG_NAME = env.PROD_MIG
                } else {
                    echo "No deployment target mapped for branch ${env.BRANCH_NAME}"
                }
            }
        }
    }

    stage('Build') {
        steps {
            sh 'mvn -B clean package -DskipTests'
        }
    }

    stage('Unit Tests') {
        steps {
            sh 'mvn -B test'
        }
        post {
            always {
                junit allowEmptyResults: true, testResults: '**/surefire-reports/*.xml'
            }
```

```
            }
        }

        stage('Static Checks') {
            steps {
                sh 'mvn -B spotbugs:check checkstyle:check || true'
            }
        }

        stage('Archive Artifact') {
            steps {
                sh "cp target/*.jar ${ARTIFACT_NAME}"
                archiveArtifacts artifacts: "${ARTIFACT_NAME}", fingerprint: true
            }
        }

        stage('Publish Release Metadata') {
            when {
                expression { env.IS_PR != 'true' && env.TARGET_ENV?.trim() }
            }
            steps {
                sh '''
                    mkdir -p release
                    cat > release/release.env <<META
APP_NAME=${APP_NAME}
BUILD_VERSION=${BUILD_VERSION}
TARGET_ENV=${TARGET_ENV}
MIG_NAME=${MIG_NAME}
META
                '''
                archiveArtifacts artifacts: 'release/release.env', fingerprint: true
            }
        }

        stage('Prod Approval') {
            when {
                allOf {
                    expression { env.IS_PR != 'true' }
                    branch 'main'
                }
            }
            steps {
                timeout(time: 30, unit: 'MINUTES') {
                    input message: 'Approve production deployment?', ok: 'Deploy to prod'
                }
            }
        }

        stage('Deploy') {
            when {
                expression { env.IS_PR != 'true' && env.TARGET_ENV?.trim() }
            }
            steps {
                script {
                    sh '''#!/bin/bash
                        set -euo pipefail

                        echo "Deploying ${APP_NAME} version ${BUILD_VERSION} to ${TARGET_ENV}"
                        echo "Example deployment flow for GCP Managed Instance Group"
                        echo "1. Upload JAR to artifact bucket or package into VM image"
                        echo "2. Create new instance template with version ${BUILD_VERSION}"
                        echo "3. Start rolling update on MIG ${MIG_NAME}"

                        # Example placeholders:
```

```
                    # gcloud auth activate-service-account --key-file=$GOOGLE_APPLICATION_CREDENTIALS
                    # gsutil cp ${ARTIFACT_NAME} gs://my-artifacts/${APP_NAME}/${BUILD_VERSION}/
                    # gcloud compute instance-templates create ${APP_NAME}-${BUILD_VERSION} ...
                    # gcloud compute instance-groups managed rolling-action start-update ${MIG_NAME} \
                    #   --version template=${APP_NAME}-${BUILD_VERSION} \
                    #   --region=${REGION} \
                    #   --max-unavailable=0 \
                    #   --max-surge=1
                '''
            }
        }
    }

    stage('Smoke Test') {
        when {
            expression { env.IS_PR != 'true' && env.TARGET_ENV?.trim() }
        }
        steps {
            sh '''#!/bin/bash
                set -euo pipefail
                echo "Running smoke tests against ${TARGET_ENV}"
                # Example:
                # curl -fsS https://sync-service-${TARGET_ENV}.example.com${HEALTH_PATH}
            '''
        }
    }
}

post {
    success {
        echo "Pipeline completed successfully for ${env.BRANCH_NAME}"
    }
    failure {
        script {
            if (env.IS_PR != 'true' && env.TARGET_ENV?.trim()) {
                echo "Deployment failed. Trigger rollback to previous stable instance template."
                sh '''#!/bin/bash
                    set +e
                    echo "Rollback placeholder: point MIG back to previous template version"
                    # Example:
                    # gcloud compute instance-groups managed rolling-action start-update ${MIG_NAME} \
                    #   --version template=${APP_NAME}-PREVIOUS_STABLE \
                    #   --region=${REGION} \
                    #   --max-unavailable=0 \
                    #   --max-surge=1
                '''
            }
        }
    }
    always {
        cleanWs(deleteDirs: true, notFailBuild: true)
    }
}
}
```